

Member customization points for Senders and Receivers

Abstract

There have been various suggestions that Senders and Receivers need a new language feature for customization points, to avoid the complexity of ADL `tag_invoke`.

This paper makes the case that C++ already has such a language facility, and it works just fine for the purposes of Senders and Receivers.

That language facility is member functions.

In a nutshell, the approach in this paper is relatively straightforward; for all non-query customization points, ADL `tag_invoke` overloads become member functions that take a customization-point-specific tag as an argument to tell them apart from possible other member functions from other libraries. Query customization points become `tag_query` member functions that take the query tag as an argument.

This is because non-queries don't need to forward calls to customization points, but it's useful for queries to be able to forward queries.

In order to be able to write perfect-forwarding function templates that work both for lvalues and rvalues, we use deduced this. When there is no need to write a single function for both lvalues and rvalues, a traditional non-static member function will do.

Quick examples

A `tag_invoke` customization point for start

```
friend void tag_invoke(std::execution::start_t, recv_op& self) noexcept
```

becomes

```
void start(std::execution::start_t) noexcept
```

A perfect-forwarding connect

```
template <__decays_to<__t> _Self, receiver _Receiver>
    requires sender_to<__copy_cvref_t<_Self, _Sender>, __receiver<_Receiver>>
    friend auto tag_invoke(std::execution::connect_t, _Self&& __self, _Receiver __rcvr)
```

becomes

```
template <__decays_to<__t> _Self, receiver _Receiver>
    requires sender_to<__copy_cvref_t<_Self, _Sender>, __receiver<_Receiver>>
    auto connect(this _Self&& __self, std::execution::connect_t, _Receiver __rcvr)
```

The call

```
tag_invoke(std::execution::connect, std::forward<Snd>(s), r);
```

becomes

```
std::forward<Snd>(s).connect(std::execution::connect, r);
```

A query

```
friend in_place_stop_token tag_invoke(std::execution::get_stop_token_t, const __t& __self) noexcept
```

becomes

```
in_place_stop_token tag_query(std::execution::get_stop_token_t) const noexcept
```

What does this buy us?

First of all, two things, both rather major:

1. NO ADL.
2. ..and that makes defining customization points **much** simpler.

A bit of elaboration on the second point: consider that earlier query of `get_stop_token` in `tag_invoke` form. It's an example of that query for the `when_all` algorithm. But what needs to be done is that both that query (which is a hidden friend) and the `when_all_t` function object type are in a detail-namespace, and then outside that namespace, in namespace `std::execution`, the type is brought into scope with a using-declaration, and the actual function object is defined.

Roughly like this:

```
namespace you_will_have_trouble_coming_up_with_a_name_for_it {
    template <class Snd, class Recv, class Fn>
    struct my_then_operation {
        opstate op;
        struct t {
            friend void tag_invoke(start_t, t& self) noexcept {
                start(self.op_);
            }
        };
    };
    // ADL-protected internal senders and receivers omitted
    struct my_then_t {
        template <sender Snd, class Fn> // proper constraints omitted
        sender auto operator(Snd&& sndr, Fn&& fn) {
            // the actual implementation omitted
        }
    };
}
using you_will_have_trouble_coming_up_with_a_name_for_it::my_then_t;
constexpr my_then_t my_then{};
```

This has the effect of keeping the overload set small, when each and every type and its customizations are meticulously defined that way. Build times are decent, the sizes of overload sets are nicely controlled and are small, diagnostics for incorrect calls are hopefully fairly okay.

But that's not all there is to it. Generic code that uses such things should wrap its template parameters into utilities that prevent ADL via template parameters. You might see something like this gem:

```
// For hiding a template type parameter from ADL
template <class _Ty>
struct _X {
    using __t = struct _T {
        using __t = _Ty;
    };
};
template <class _Ty>
using __x = __t<_X<_Ty>>;
```

and then use it like this:

```
using make_stream_env_t = stream_env<stdexec::__x<BaseEnv>>;
```

With member customization points, you don't need any such acrobatics. The customization points are members. You define a customization point as a member function, and you can just put your type directly into whichever namespace you want (some might even use the global namespace), and you don't need to use nested detail namespaces. Then you call `foo.connect(std::execution::connect, receiver);` and you don't have to do the no-ADL wrapping in your template parameters either.

The definition of customization points is much simpler, to a ridiculous extent. Using them is simpler; it's a member call, everybody knows what that does, and many people know what scopes that looks in, and a decent amount of people appreciate the many scopes it **doesn't** look in.

Composition and reuse and wrapping of customization points becomes much easier, because it's just.. ..good old OOP, if you want to look at it that way. We're not introducing a new language facility for which you need to figure out how to express various function compositions and such, the techniques and patterns are decades old, and work here as they always worked.

What are its downsides compared to a new language facility?

Well, we don't do anything for users who for some reason *_have_* to use ADL customization points. But the reason for going for this approach is that we decouple Senders and Receivers from an unknown quantity, and avoid many or even most of the problems of using ADL customization points.

Other than that, I'm not sure such downsides exist.

A common concern with using wrappers is that they don't work if you have existing APIs that use the wrappees - introducing wrappers into such situations just doesn't work, because they simply aren't the same type, and can't be made the same type. And a further problem is having to deal with both the wrappers and wrappees as concrete types, and figuring out when to use which, and possibly having to duplicate code to deal with both.

The saving grace with Senders and Receivers is that they are wrapped everywhere all the time. Algorithms wrap senders, the wrapped senders wrap their receivers, and resulting operation states. This wrapping nests pretty much infinitely.

For cases where you need to use a concrete sender, it's probably type-erased, rather than being a use of a concrete target sender.

Implementation experience

A partial work-in-progress implementation exists as a branch of the reference implementation of P2300, at https://github.com/villevoutilainen/wg21_p2300_std_execution/tree/member-only-customization-points

The implementation macro-migrates from ADL `tag_invoke` overloads to static member functions and member functions using deduced this. This is done to provide existing users a migration path.

Some additional considerations

Access

It's possible to make customization point members private, and have them usable by the framework, by befriending the entry point (e.g. `std::execution::connect`, in a member `connect(std::execution::connect_t)`). It's perhaps ostensibly rare to need to do that, considering that it's somewhat unlikely that a sender wrapper or an operation state wrapper that provides the customization point would have oodles of other functionality. Nevertheless, we have made that possible in the prototype implementation, so we could do the same in the standard. This seems like an improvement over the ADL customization points. With them, anyone can do the ADL call, the access of a hidden friend doesn't matter.

Interface pollution

It's sometimes plausible that a class with a member customization point inherits another class that provides the same customization point, and it's not an override of a virtual function. In such situations, the traditional technique works, bring in the base customization point via a using-declaration, which silences possible hiding warnings and also create an overload set. The expectation is that the situation and the technique are sufficiently well-known, since it's old-skool.

Wording

There is no specification change yet that would show what P2300 looks like with this approach applied. The expectation is that that change is actually fairly straightforward. The code changes certainly are, although the macro-wrappings make it less readable than the end result is.